

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Комп'ютерна логіка. Частина 2

Лабораторна робота №5

«Дослідження операцій з числами у форматі з плаваючою комою»

Виконав: студент 1 курсу ФІОТ, гр. ІО-41

Давидчук А. М.

Залікова книжка № 4106

Перевірив: *Верба О. А.*

Тема: «Дослідження операцій з числами у форматі з плаваючою комою».

Мета: Дослідити машинні методи виконання операцій з числами в форматі з плаваючою комою.

Виконання роботи

Мій номер залікової книжки: 4106, що у двіковому вигляді є 0001 0000 0000 1010. Звідси $x_9 = 0, x_8 = 0, x_7 = 0, x_6 = 0, x_5 = 0, x_4 = 1, x_3 = 0, x_2 = 1, x_1 = 0$. Звідси визначу два числа X та Y :

$$X = -x_7x_61x_5x_40, x_31x_2x_1 = -001010, 0110,$$

$$Y = +x_91x_8x_7, x_6x_5x_4x_3x_2x_1 = +0100, 001010$$

Також визначу способи виконання двох операцій множення, однієї операції ділення та додавання:

x_2, x_1	Спосіб множення	x_3	Спосіб ділення	x_5, x_4	Машинні коди в АЛУ
00	1-й, 4-й	0	1-й	00	Додавання в ОК
01	2-й, 3-й	1	2-й	01	Додавання в ДК
10	1-й, 3-й		1-й	10	Віднімання в ОК
11	2-й, 4-й		2-й	11	Віднімання в ДК

Тобто у мене 1-й та 3-й спосіб множення, 1-й спосіб ділення та додавання в ДК.

Тепер подам числа X та Y у форматі з плаваючою комою, шляхом переміщення «фіксованої» коми перед старшим бітом мантиси, і відповідним значенням порядку (для порядку я виділю 4 біти, що буде достатнім для цієї лабораторної роботи):

$$X = \boxed{0} \boxed{0110} \boxed{1} \boxed{0010100110}$$

$$Y = \boxed{0} \boxed{0100} \boxed{0} \boxed{0100001010}$$

Так як мантиса подається у вигляді [знак; модуль], то значить ми можемо говорити про нормалізацію мантис як додатніх мантис, які у старшому розряді завжди мають 1. Але як видно, мантиси чисел X та Y не задовільняють нормалізованого вигляду, тому проведемо нормалізацію мантис шляхом зсуву вліво на n кількість бітів і віднімання порядку на n , поки не буде досягнуто нормалізованої мантиси:

$$X = \boxed{0} \boxed{0100} \boxed{1} \boxed{1010011000}$$

$$Y = \boxed{0} \boxed{0011} \boxed{0} \boxed{1000010100}$$

Тепер подам числа X та Y за стандартом ANSI/IEEE 754-2008 в короткому 32-бітному форматі. Для цього потрібно знайти зміщений порядок (8 бітів), а також розширити мантису до 23 бітів:

$$X : P_{\text{зміщ}} = P + (2^{8-1} - 1)_{10} = P + 127_{10} = 4_{10} + 127_{10} = 131_{10} = 10000011_2, \text{ тоді}$$

$$X_{\text{к. ф.}} = 1.10000011, 101001100000000000000000$$

$$Y : P_{\text{зміщ}} = P + (2^{8-1} - 1)_{10} = P + 127_{10} = 3_{10} + 127_{10} = 130_{10} = 10000010_2, \text{ тоді}$$

$$Y_{\text{к. ф.}} = 0.10000010, 100001010000000000000000$$

Перейдімо до виконання операцій над числами у форматі з плаваючою комою.

Але перш ніж перейти вже до виконання операцій, ми маємо узгодити розрядність мантис та операційних пристроїв. В даній лабораторній роботі на мантису відводиться 7 розрядів (з урахуванням знакового) — тому я округлю мантису X до 6 розрядів (без урахування знакового):

$$\begin{array}{r} M_X : 1010011 \\ + 0000001 \\ \hline 1010100 \end{array}$$

Тобто $M_X \approx 101010$, а $M_Y \approx 100001$. Тоді

$$\begin{array}{l} X = \boxed{0} \quad \boxed{0100} \quad \boxed{1} \quad \boxed{101010} \\ Y = \boxed{0} \quad \boxed{0011} \quad \boxed{0} \quad \boxed{100001} \end{array}$$

Множення:

Теоретичне підґрунтя:

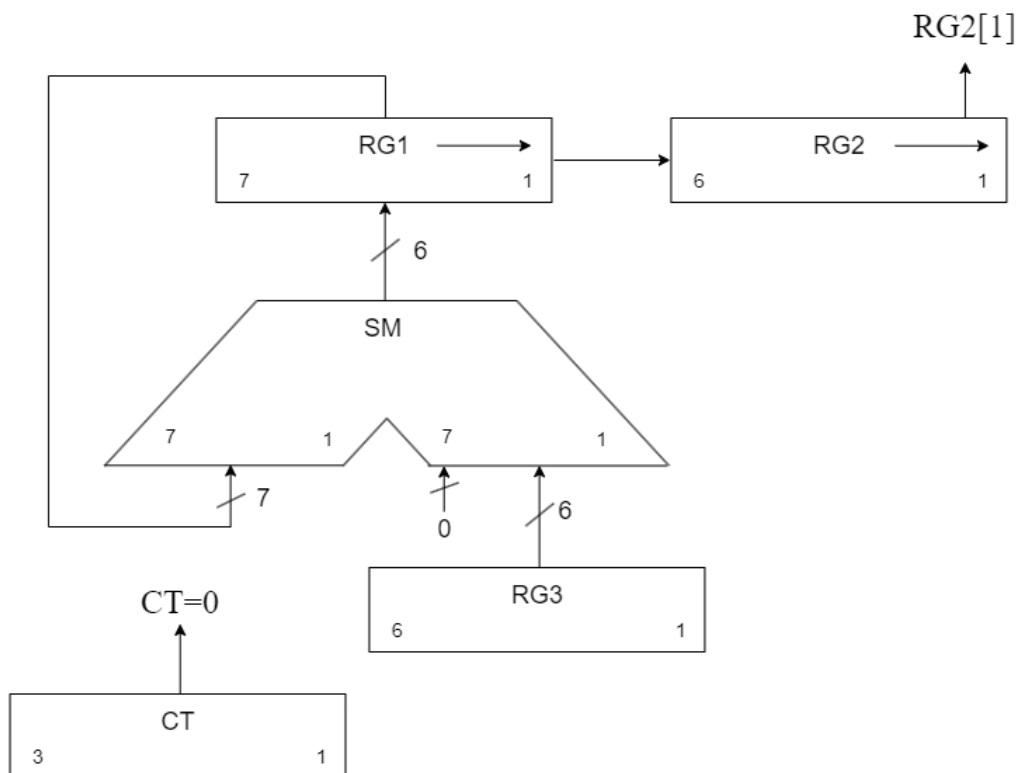
Дано два числа: $X = 2^a M_X$ та $Y = 2^b M_Y$.

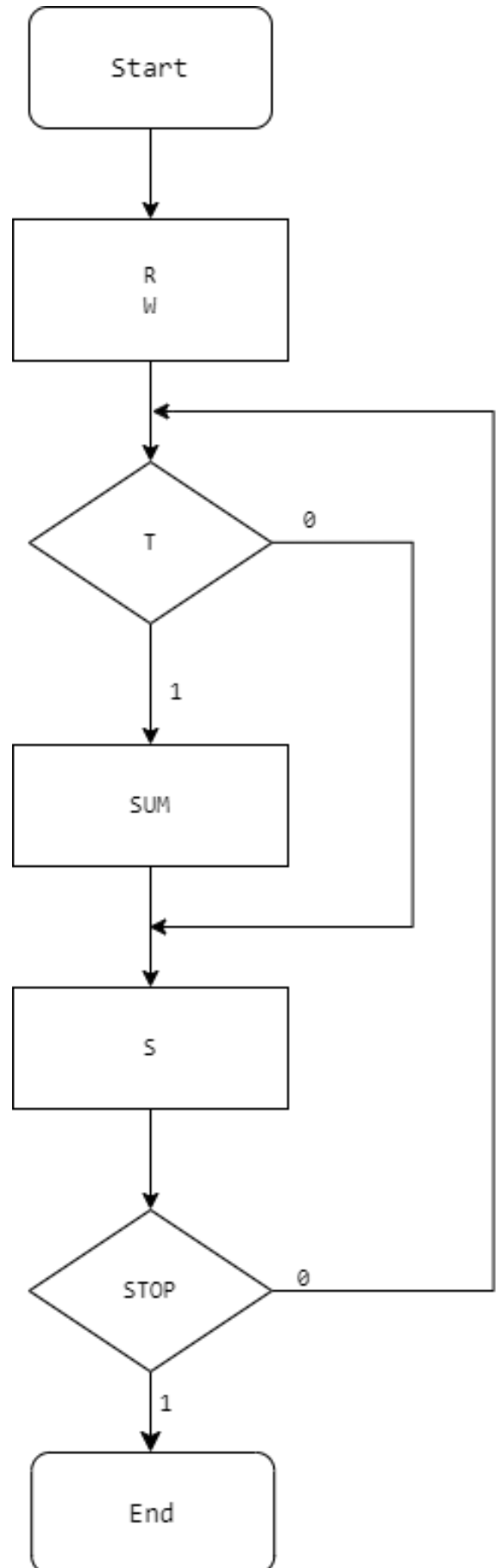
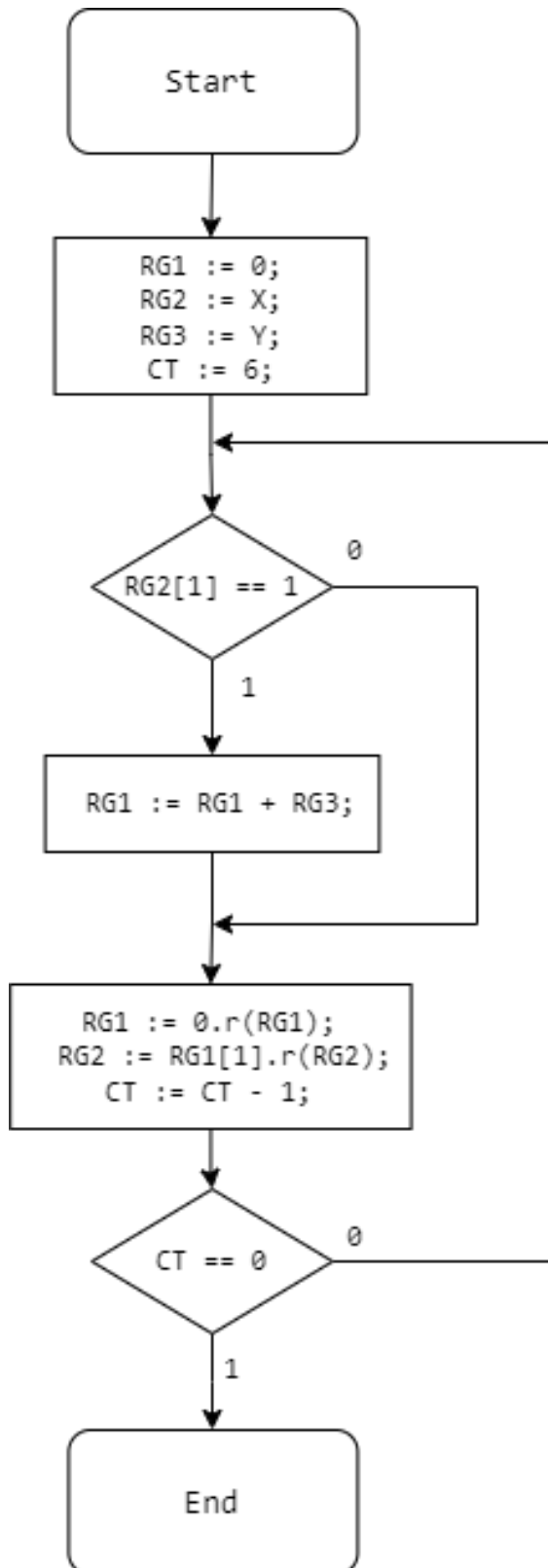
Під час множення двох чисел отримаємо: $X \cdot Y = 2^a M_X \cdot 2^b M_Y = 2^{a+b} (M_X \cdot M_Y)$.

Тобто при множенні двох чисел у форматі з плаваючою комою: додаються порядки, множаться мантиси цих чисел.

Множення 1-й спосіб:

Операційна схема множення 1-го способу для 6 розрядних чисел:





Таблиця станів регістрів:

$$M_Z = M_X \cdot M_Y = ,101010 \times ,100001 :$$

№	RG1 7 1	RG2 6 1	RG3 7 1	CT 3 1	Мікрооперації
—	0000000	101010	0100001	110	RG1 = 0, RG2 = X, RG3 = 0.Y; CT = n = 6;
1	0000000 <i>Після зсуву:</i> 0000000	101010 <i>Після зсуву:</i> 010101		101	RG1 = 0.r[RG1], RG2 = RG1[1].r[RG2], CT = CT - 1;
2	+0000000 0100001 — 0100001 <i>Після зсуву:</i> 0010000	010101 <i>Після зсуву:</i> 101010		100	RG1 = RG1 + RG3; RG1 = 0.r[RG1], RG2 = RG1[1].r[RG2], CT = CT - 1;
3	0010000 <i>Після зсуву:</i> 0001000	101010 <i>Після зсуву:</i> 010101		011	RG1 = 0.r[RG1], RG2 = RG1[1].r[RG2], CT = CT - 1;
4	+0001000 0100001 — 0101001 <i>Після зсуву:</i> 0010100	010101 <i>Після зсуву:</i> 101010		010	RG1 = RG1 + RG3; RG1 = 0.r[RG1], RG2 = RG1[1].r[RG2], CT = CT - 1;
5	0010100 <i>Після зсуву:</i> 0001010	101010 <i>Після зсуву:</i> 010101		001	RG1 = 0.r[RG1], RG2 = RG1[1].r[RG2], CT = CT - 1;
6	+0001010 0100001 — 0101011 <i>Після зсуву:</i> 0010101	010101 <i>Після зсуву:</i> 101010		000	RG1 = RG1 + RG3; RG1 = 0.r[RG1], RG2 = RG1[1].r[RG2], CT = CT - 1;

Знаковий біт мантиси $M_Z : 1 \oplus 0 = 1$ (число від'ємне). Тобто $M_Z = 1,010101101010$, а $P_Z = P_X + P_Y = 4_{10} + 3_{10} = 7_{10} = 0111_2$. Але старший біт (без знакового) мантиси M_Z не дорівнює 1, тому нормалізуємо, і отримаємо:

$$M_Z = 1,10101101010, P_Z = 0110.$$

Також звернемо увагу, що потрібно виділити 6 бітів (без знакового) в мантисі, тому потрібно виконати округлення до 6 розрядів. Помітимо, що 7 біт мантиси = 0, це значить, що округлення можна провести шляхом відкидання бітів, окрім перших 6. Тобто, $M_Z \approx 1,101011, P_Z = 0110$. Звідси

$$Z = \boxed{0} \boxed{0110} \boxed{1} \boxed{101011}$$

Функціональна схема множення, в якій використовується 8-бітний регістр та суматор для зручності:

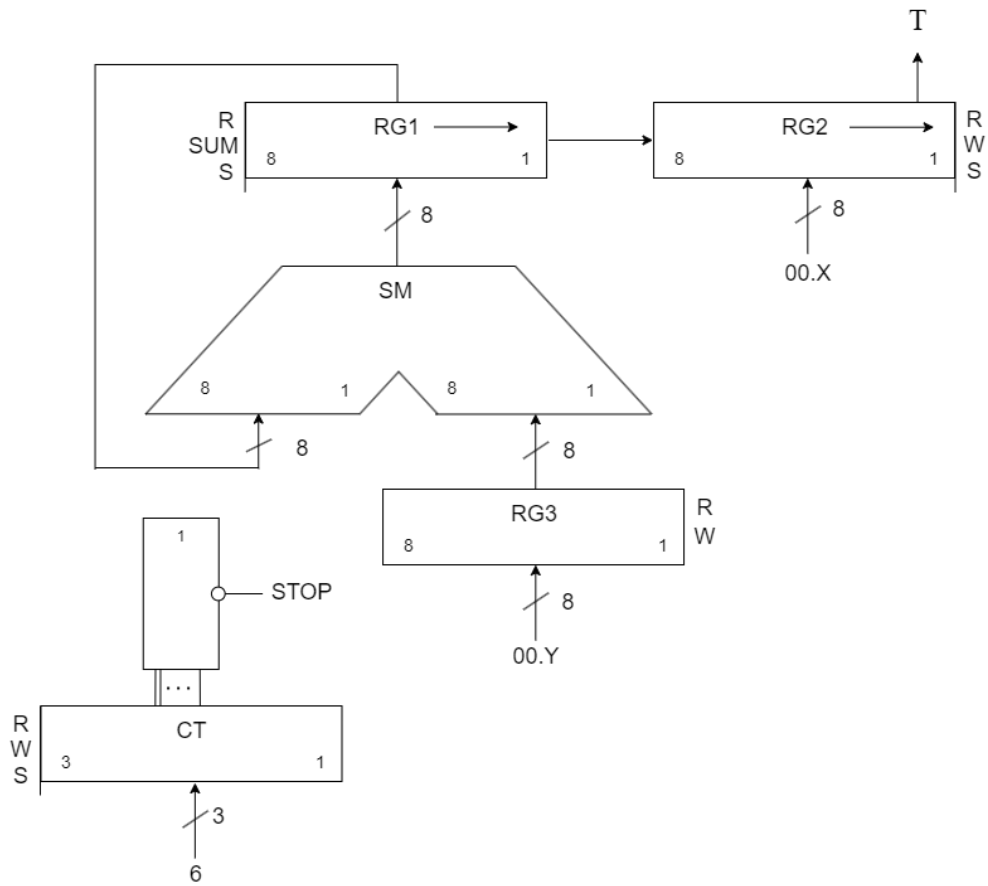
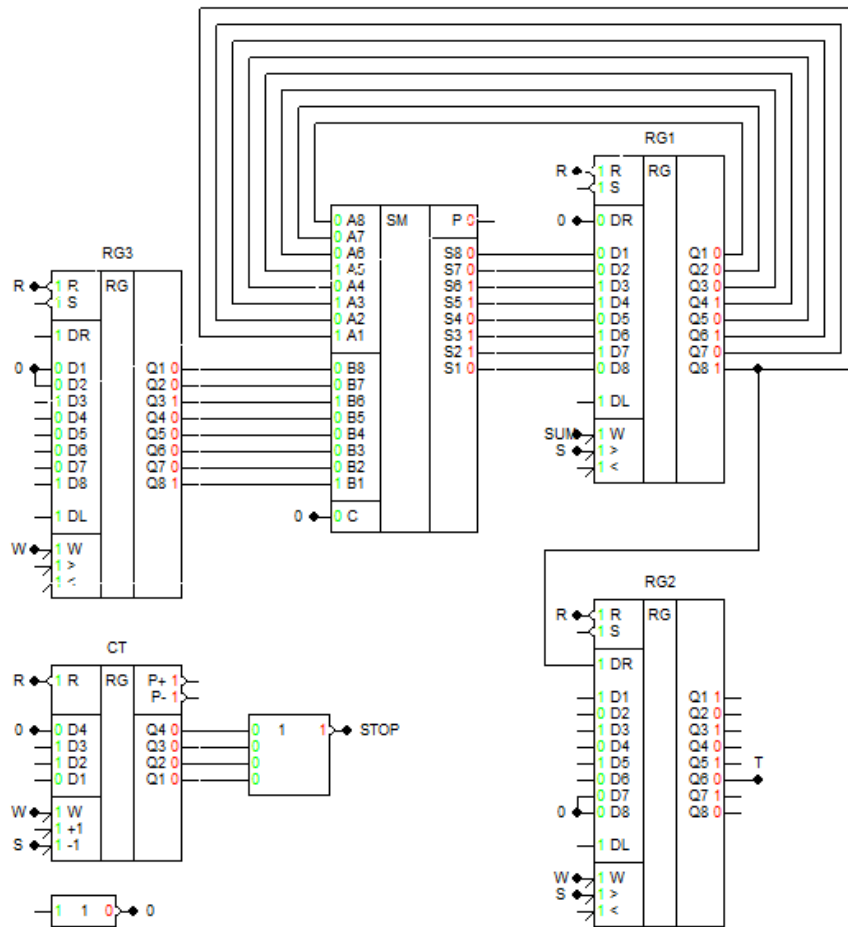
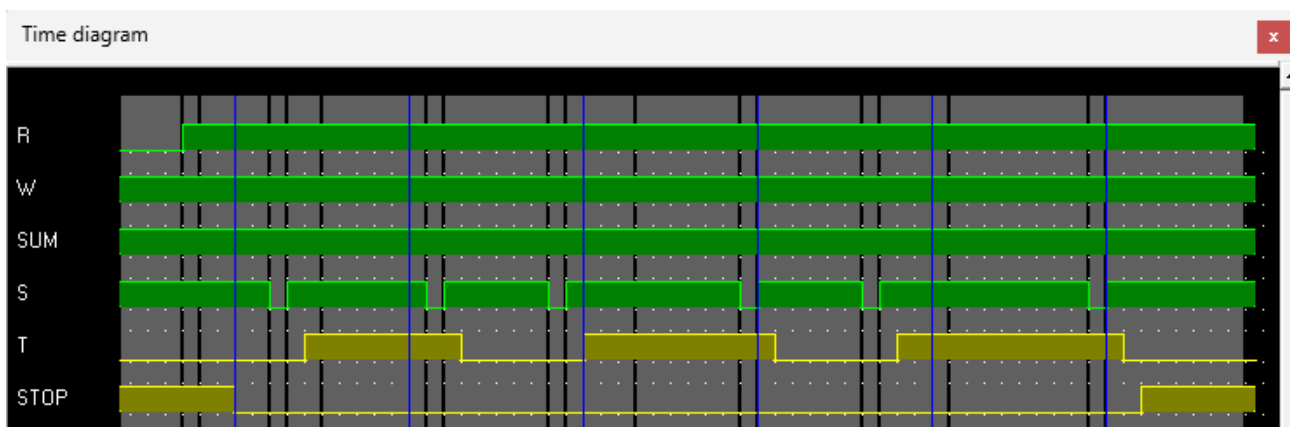


Схема AFDK:



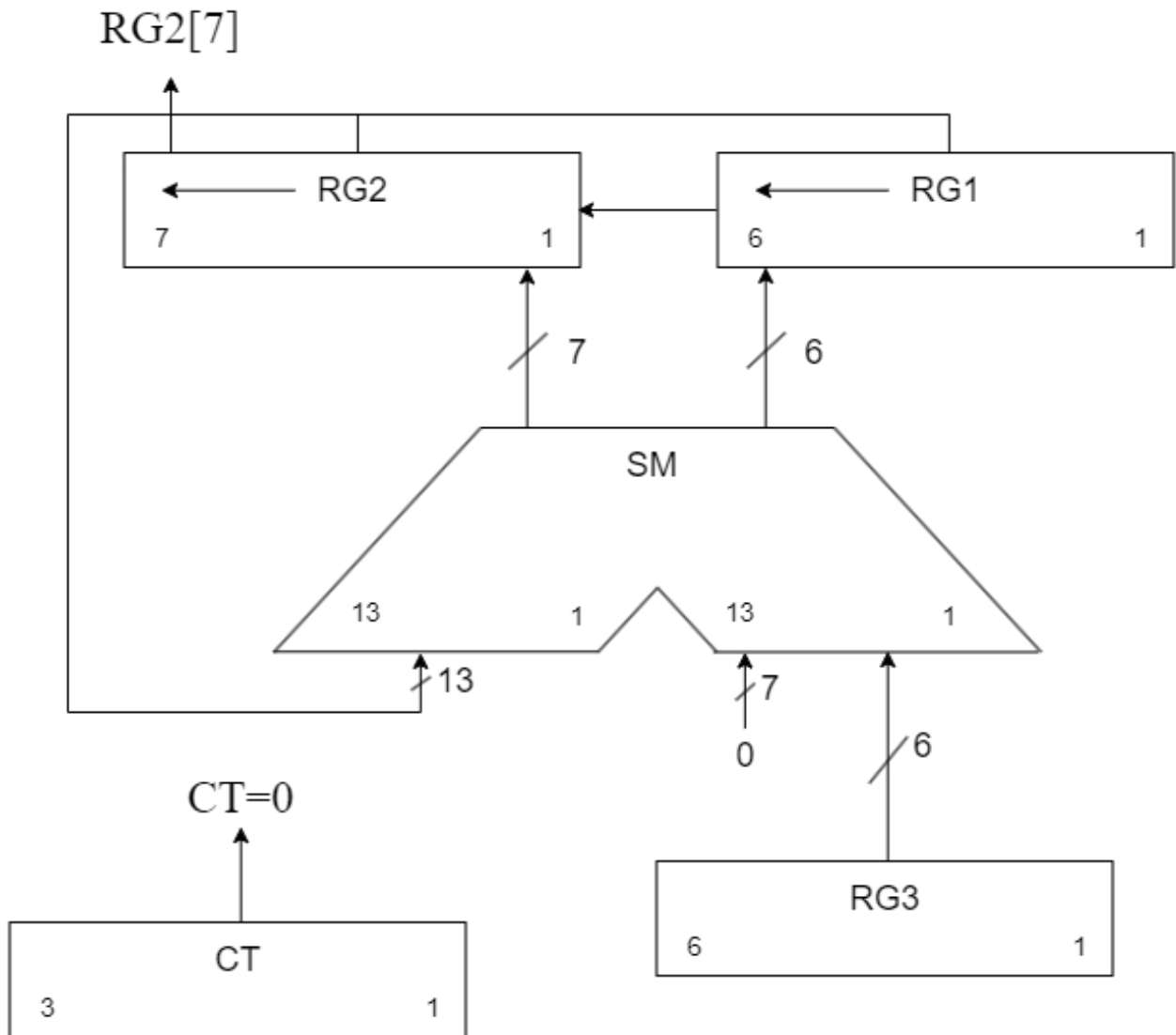
Часова діаграма:



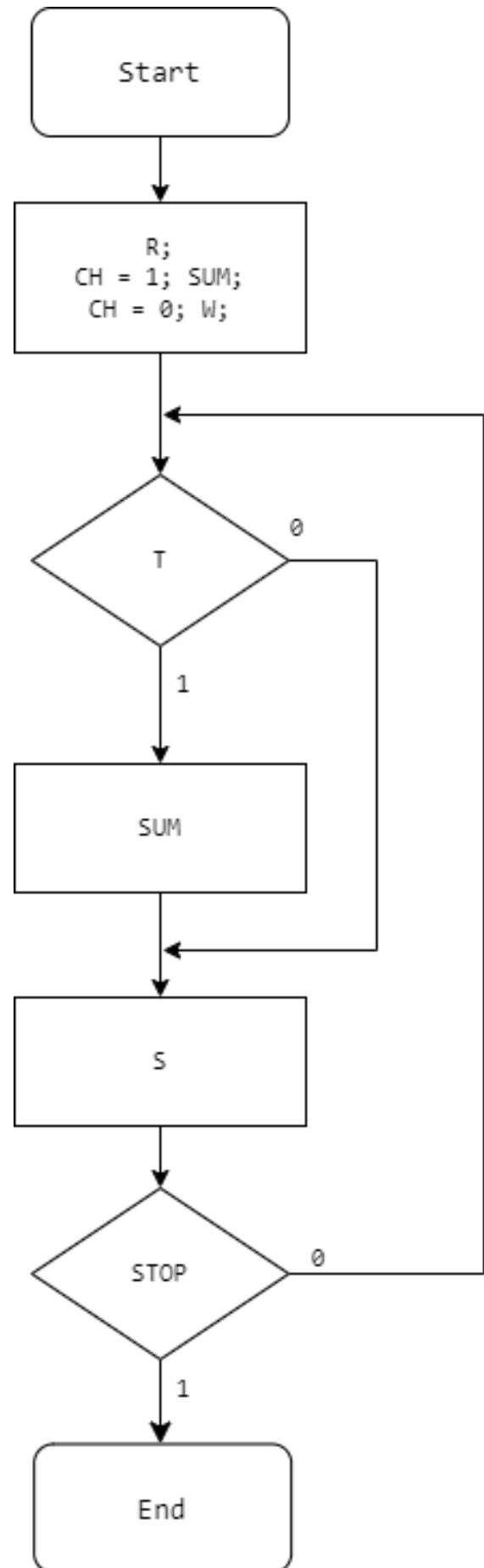
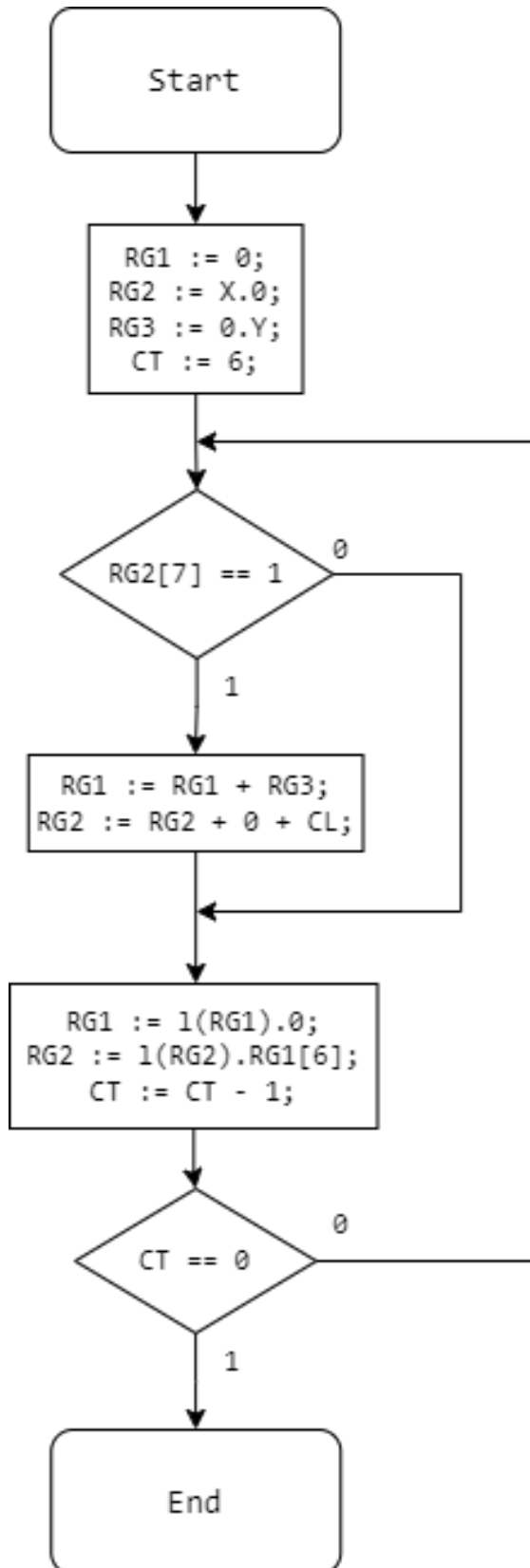
Результат множення збігається з попередньо обчисленим.

Множення 3-й спосіб:

Операційна схема множення 3-го способу для 6 розрядних чисел:



Змістовний та ФС мікроалгоритми:



Таблиця станів регістрів:

$$M_N = M_Z \cdot M_Y = ,101011 \times ,100001 :$$

№	RG2 7 1	RG1 6 1	RG3 6 1	CT 3 1	Мікрооперації
—	1010110	000000	100001	110	RG3 := 0.Y, RG2 := X.0, RG1 := 0; CT := n := 6; CT = CT - 1;
1	1010110 <i>Після зсуву:</i> 0101101	+000000 100001 — 100001 <i>Після зсуву:</i> 000010		101	RG1 := RG1 + RG3, RG2 := RG2 + 0 + CI; RG1 := 1(RG1).0, RG2 := 1(RG2).RG1[6]; CT = CT - 1;
2	0101101 <i>Після зсуву:</i> 1011010	000010 <i>Після зсуву:</i> 000100		100	RG1 := 1(RG1).0, RG2 := 1(RG2).RG1[6]; CT = CT - 1;
3	1011010 <i>Після зсуву:</i> 0110101	+000100 100001 — 100101 <i>Після зсуву:</i> 001010		011	RG1 := RG1 + RG3, RG2 := RG2 + 0 + CI; RG1 := 1(RG1).0, RG2 := 1(RG2).RG1[6]; CT = CT - 1;
4	0110101 <i>Після зсуву:</i> 1101010	001010 <i>Після зсуву:</i> 010100		010	RG1 := 1(RG1).0, RG2 := 1(RG2).RG1[6]; CT = CT - 1;
5	1101010 <i>Після зсуву:</i> 1010101	+010100 100001 — 110101 <i>Після зсуву:</i> 101010		001	RG1 := RG1 + RG3, RG2 := RG2 + 0 + CI; RG1 := 1(RG1).0, RG2 := 1(RG2).RG1[6]; CT = CT - 1;
6	+1010101 0000001 — 1010110 <i>Після зсуву:</i> 0101100	+101010 100001 — 001011 <i>Після зсуву:</i> 010110		000	RG1 := RG1 + RG3, RG2 := RG2 + 0 + CI; RG1 := 1(RG1).0, RG2 := 1(RG2).RG1[6]; CT = CT - 1;

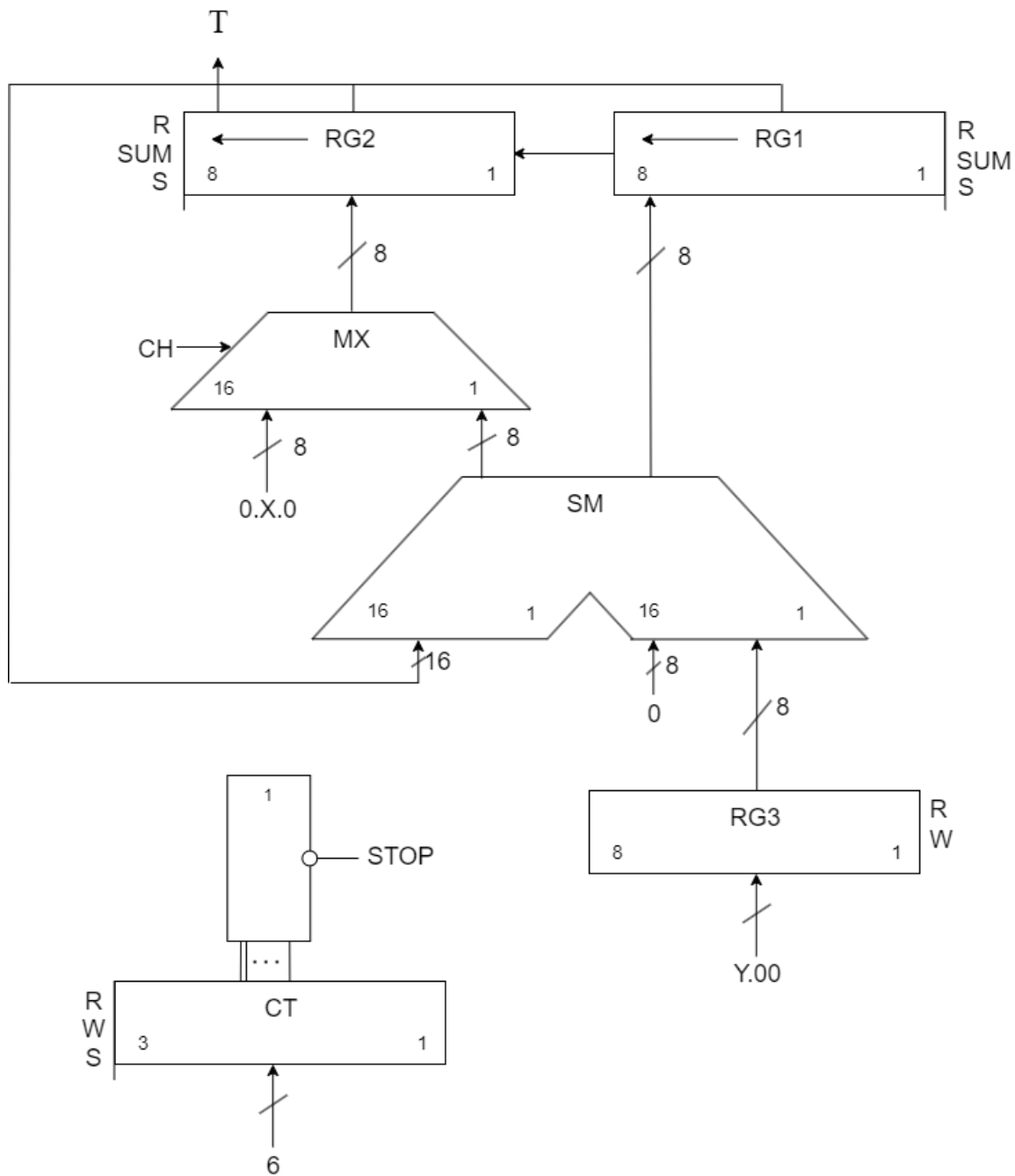
Знаковий біт мантиси $M_N : 1 \oplus 0 = 1$ (число від'ємне). Тобто $M_N = 1,010110001011$, а $P_N = P_Z + P_Y = 6_{10} + 3_{10} = 9_{10} = 1001_2$. Але старший біт (без знакового) мантиси M_N не дорівнює 1, тому нормалізуємо, і отримаємо:

$$M_N = 1,101100010110, P_N = 1000.$$

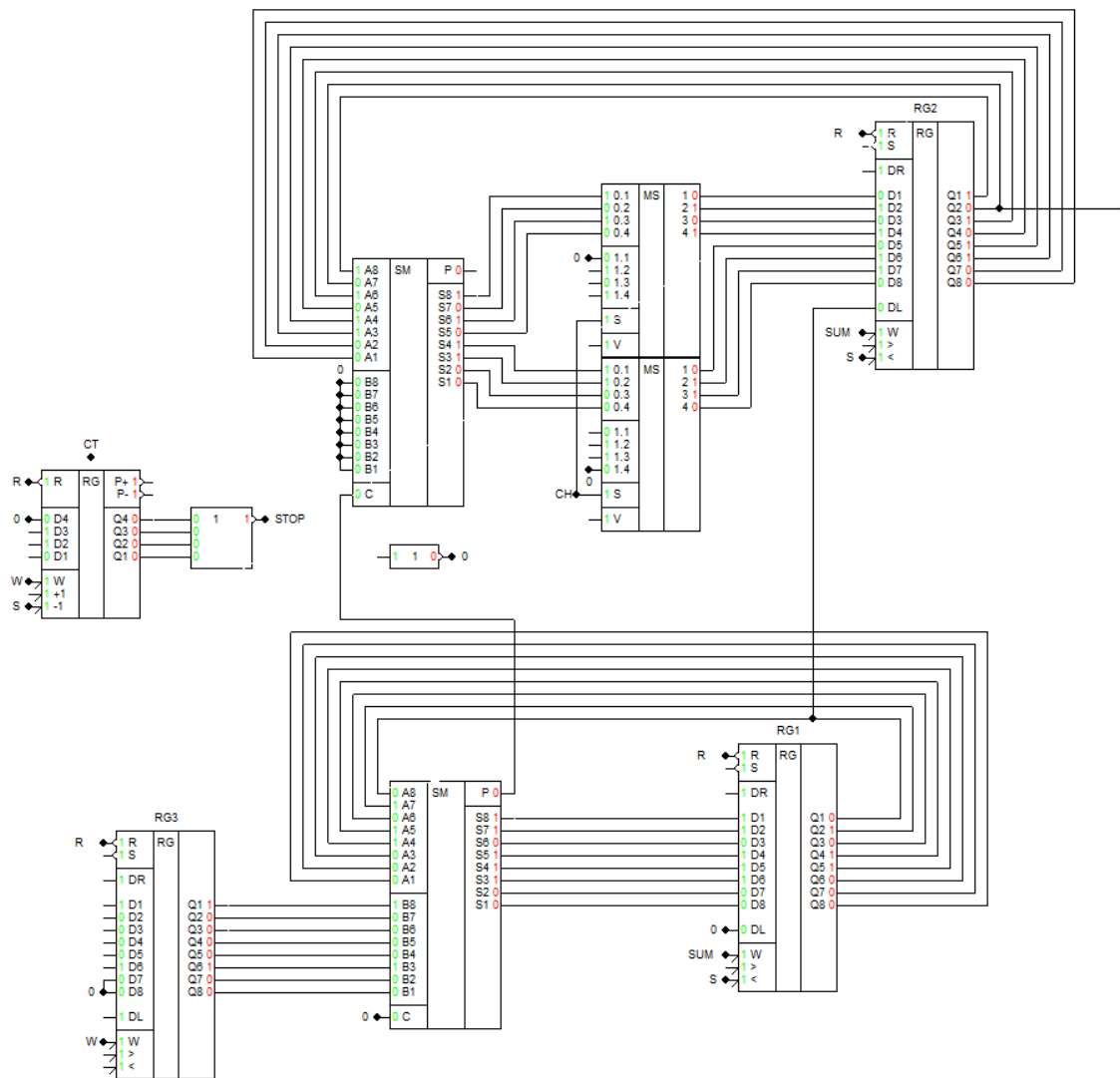
Також звернемо увагу, що потрібно виділити 6 бітів (без знакового) в мантисі, тому потрібно виконати округлення до 6 розрядів. Помітимо, що 7 біт мантиси = 0, це значить, що округлення можна провести шляхом відкидання бітів, окрім перших 6. Тобто, $M_N \approx 1,101100$, $P_N = 1000$. Звідси

$$N = \boxed{0} \quad \boxed{1000} \quad \boxed{1} \quad \boxed{101100}$$

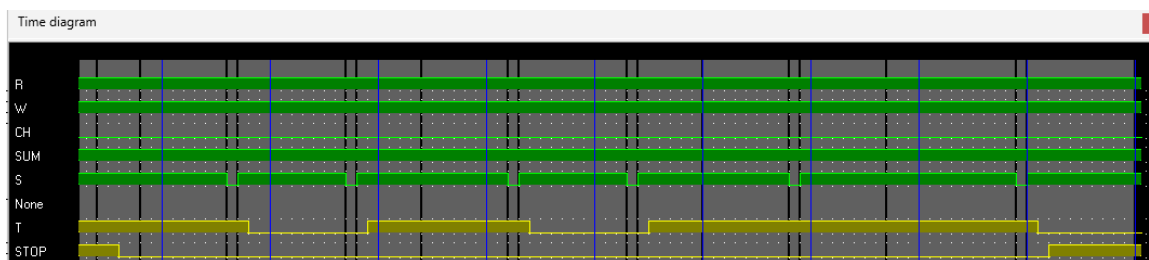
Функціональна схема множення, в якій використовується 8-бітний регістр, суматор та мультимплексор для зручності, та в подальшому масштабуванні:



AFDK:



Часова діаграма:



Результат множення збігається з попередньо обчисленим.

Ділення:

Теоретичне підґрунтя:

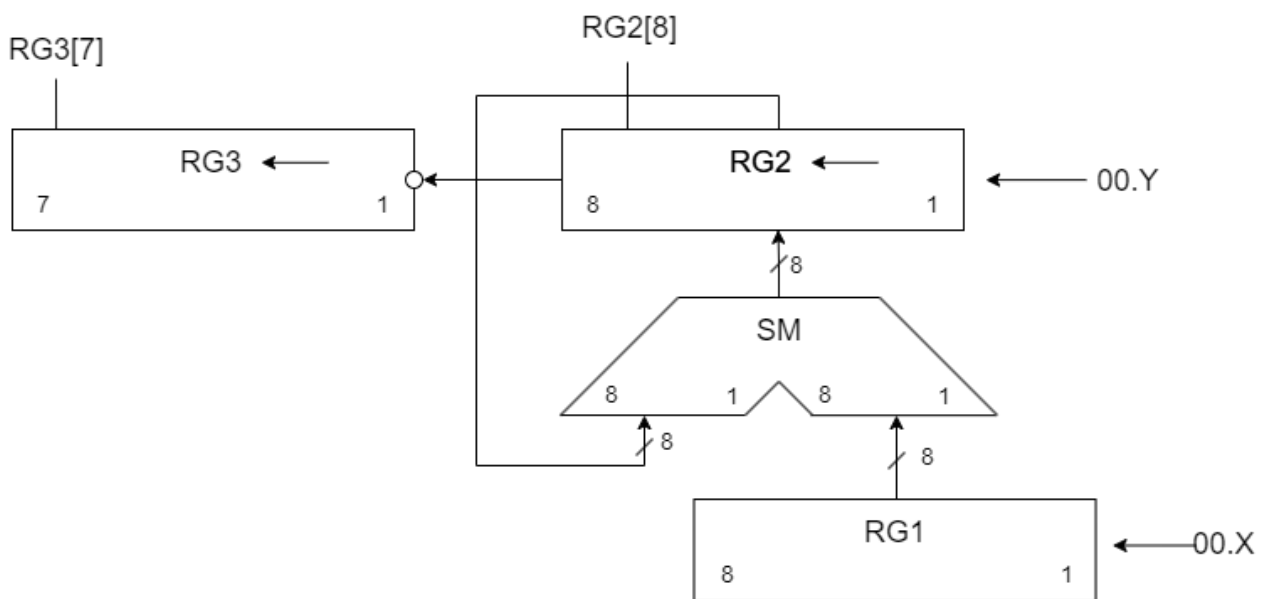
Дано два числа: $X = 2^a M_X$ та $Y = 2^b M_Y$.

Під час ділення двох чисел отримаємо: $X/Y = 2^a M_X / 2^b M_Y = 2^{a-b} (M_X/M_Y)$.

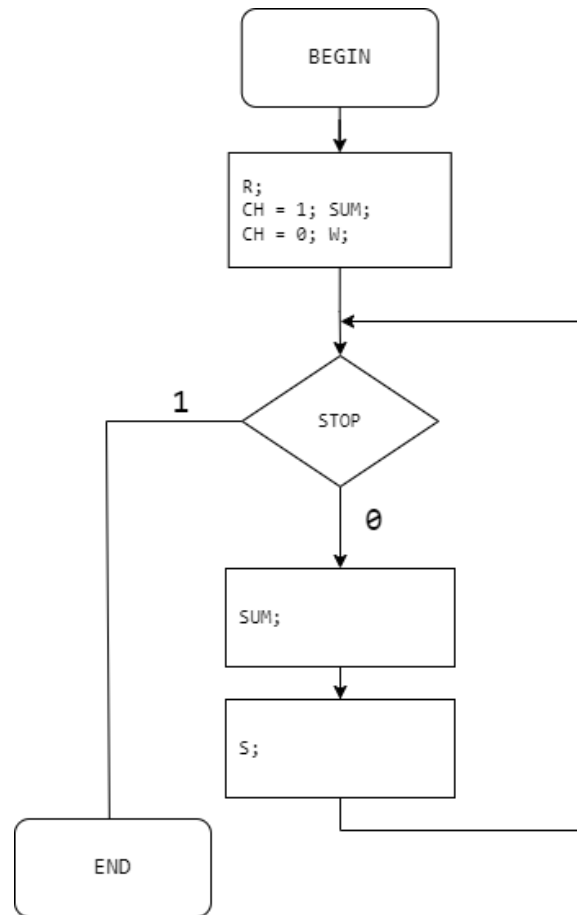
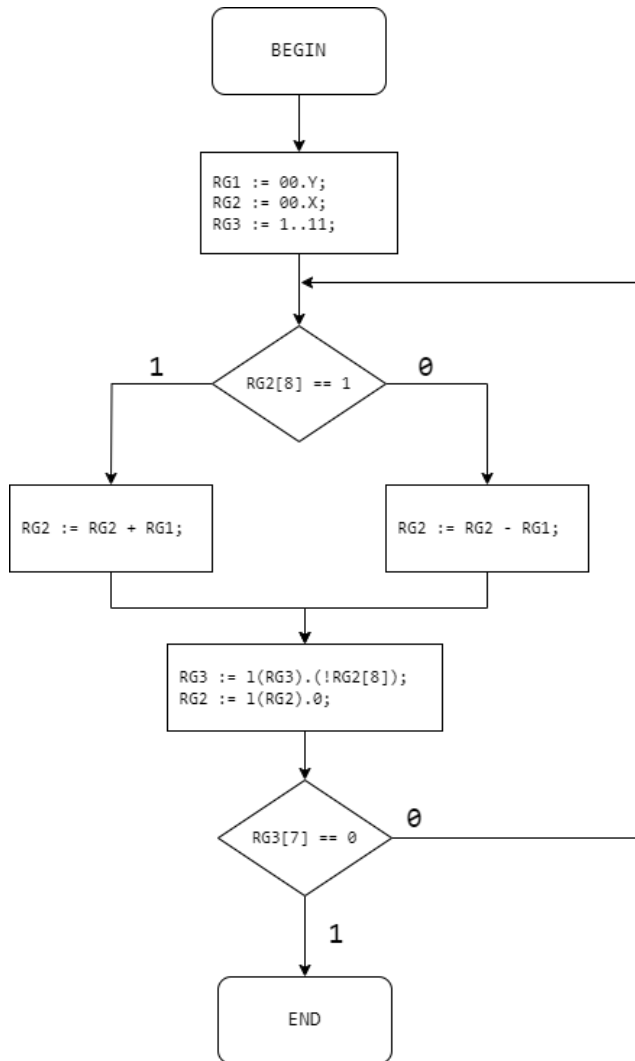
Тобто при діленні двох чисел у форматі з плаваючою комою: віднімаються порядки, діляться мантиси цих чисел. Ділення можна виконати, за умови $M_X < M_Y$. Якщо така умова не є істинною від початку, тоді зсовуємо мантису M_X вправо, та додаємо 1 до значення порядку числа X .

Ділення 1-й спосіб:

Операційна схема множення 3-го способу для 6 розрядних чисел:



Змістовний та ФС мікроалгоритми:



Таблиця станів регістрів:

$M_F = M_N/M_Y = ,101100 / ,100001$. Звідси бачимо, що умова $M_N < M_Y$ НЕ виконується, тому зсунемо мантису M_N на 1 біт вправо, і додамо 1 до порядку P_N . Тим самим, отримуємо $M_N = ,010110$, $P_N = 0111$. Тут же виконується умова $M_N < M_Y$.

№	RG3 71	RG2 81	RG1 81	Мікрооперації
—	1111111	00010110	$RG1 = 00100001$ $-RG1 + 1$ (від’ємне число $00.Y$ в ДК): $+11011110$ 00000001 11011111	$RG3 = 1..1$; $RG2 = 00.X$; $RG1 = 00.Y$;
1	1111111 <i>Після зсуву:</i> 1111110	$+00010110$ 11011111 11110101 <i>Після зсуву:</i> 11101010		$RG2 = RG2 + (-RG1) + D$; $RG3 = 1(RG3).(!RG2[8])$; $RG2 = 1(RG2).0$;
2	1111110 <i>Після зсуву:</i> 1111101	$+11101010$ 00100001 00001011 <i>Після зсуву:</i> 00010110		$RG2 = RG2 + RG1$; $RG3 = 1(RG3).(!RG2[8])$; $RG2 = 1(RG2).0$;
3	1111101 <i>Після зсуву:</i> 1111010	$+00010110$ 11011111 11110101 <i>Після зсуву:</i> 11101010		$RG2 = RG2 + (-RG1) + D$; $RG3 = 1(RG3).(!RG2[8])$; $RG2 = 1(RG2).0$;
4	1111010 <i>Після зсуву:</i> 1110101	$+11101010$ 00100001 00001011 <i>Після зсуву:</i> 00010110		$RG2 = RG2 + RG1$; $RG3 = 1(RG3).(!RG2[8])$; $RG2 = 1(RG2).0$;
5	1110101 <i>Після зсуву:</i> 1101010	$+00010110$ 11011111 11110101 <i>Після зсуву:</i> 11101010		$RG2 = RG2 + (-RG1) + D$; $RG3 = 1(RG3).(!RG2[8])$; $RG2 = 1(RG2).0$;
6	1101010 <i>Після зсуву:</i> 1010101	$+11101010$ 00100001 00001011 <i>Після зсуву:</i> 00010110		$RG2 = RG2 + RG1$; $RG3 = 1(RG3).(!RG2[8])$; $RG2 = 1(RG2).0$;
7	1010101 <i>Після зсуву:</i> 0101010	$+00010110$ 11011111 11110101 <i>Після зсуву:</i> 11101010		$RG2 = RG2 + (-RG1) + D$; $RG3 = 1(RG3).(!RG2[8])$; $RG2 = 1(RG2).0$;

Знаковий біт мантиси $M_F : 1 \oplus 0 = 1$ (число від’ємне). Тобто $M_F = 1,10101011101010$, а $P_F = P_N + P_Y = 7_{10} + 3_{10} = 10_{10} = 1010_2$. Як бачимо, старший біт мантиси є 1, тому нормалізація не потрібна. Тобто:

$M_F = 1,10101011101010$, $P_F = 1010$.

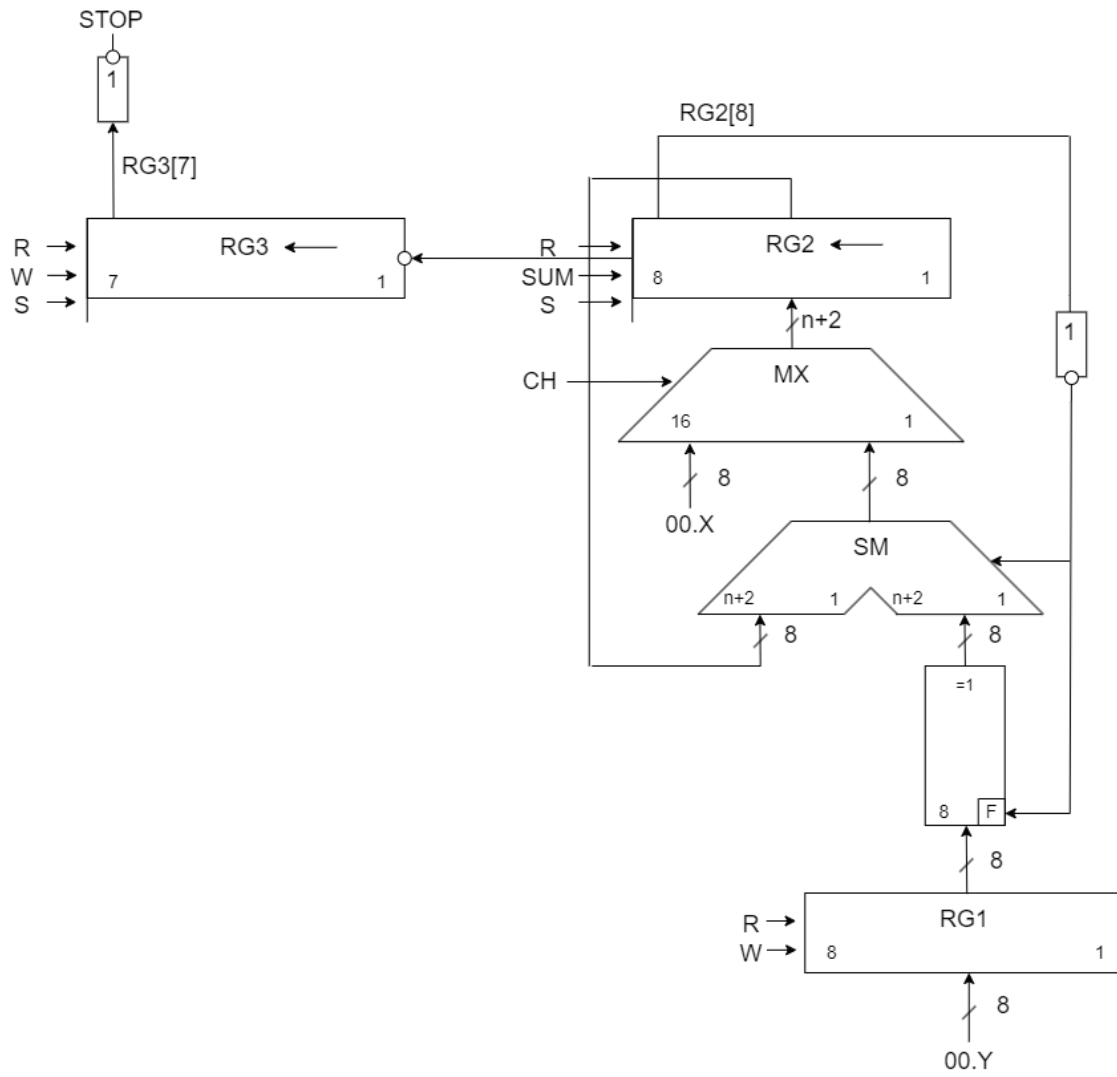
Також звернемо увагу, що потрібно виділити 6 бітів (без знакового) в мантисі, тому потрібно виконати округлення до 6 розрядів. Помітимо, що 7 біт мантиси = 1, це значить, що округлення можна провести шляхом додавання одиниці до 7 біту із подальшим переносом одиниць. Тобто,

$$\begin{array}{r} M_F : 1010101 \\ + 0000001 \\ \hline 1010110 \end{array}$$

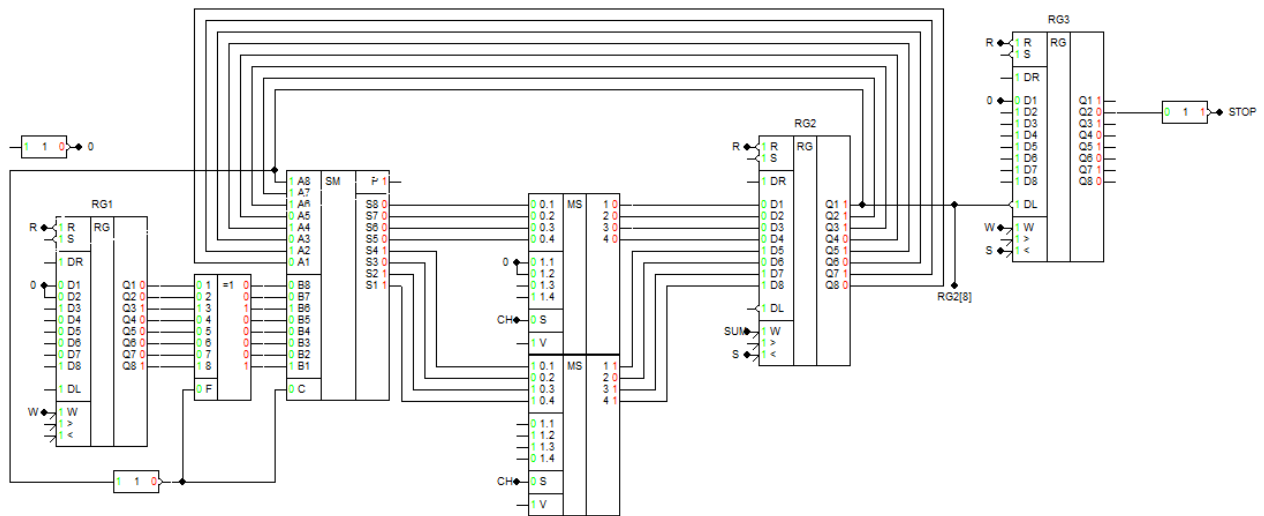
Звідси $M_F \approx 1,101011$, $P_F = 1010$, Або

$$F = \boxed{0} \quad \boxed{1010} \quad \boxed{1} \quad \boxed{101011}$$

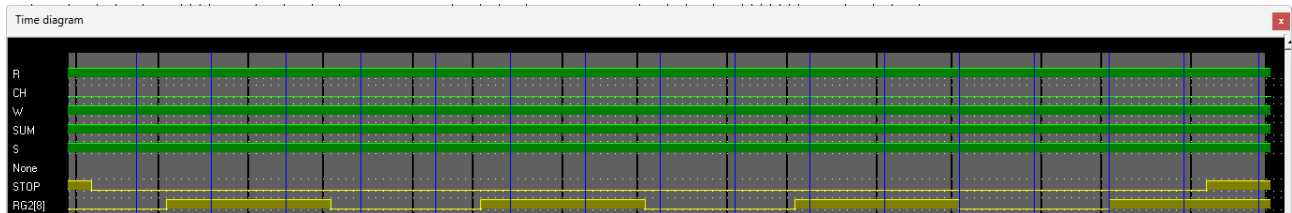
Функціональна схема ділення:



AFDK:



Часова діаграма:



Результат ділення збігається з попередньо обчисленим.

Додавання:

Теоретичне підґрунтя:

Дано два числа: $X = 2^a M_X$ та $Y = 2^b M_Y$.

Під час додавання двох чисел отримаємо: $X + Y = 2^a M_X + 2^b M_Y$.

Щоб провести додавання мантий, потрібно домогтись того, щоб порядки були однакові. Зазвичай обирають опорним порядком є більший порядок. Щоб звести менший порядок до більшого, потрібно знайти їх різницю в десятковому представленні. І настільки ж зсунути мантию меншого порядку вправо. Після цього можна виконувати додання/віднімання. Як результативний порядок приймаємо найбільший.

Додавання ДК:

У нас є два нормалізованих числа у форматі з плаваючою комою:

$$\begin{array}{l} F = \boxed{0} \quad \boxed{1010} \quad \boxed{1} \quad \boxed{101011} \\ Y = \boxed{0} \quad \boxed{0011} \quad \boxed{0} \quad \boxed{100001} \end{array}$$

Звідси $P_F > P_Y$, тоді визнаємо порядок P_F опорним. Кількість зсувів: $P_F - P_Y = 10_{10} - 3_{10} = 7_{10}$. Тобто мантию числа Y потрібно зсунути вправо на 7 бітів:

№ зсуву	Результат зсуву M_Y
—	100001
1	010000
2	001000
3	000100
4	000010
5	000001
6	000000
7	000000

Значить додавання мантий в ДК буде (зادля спрощення запису, приймемо, що $R_{\text{ДК}}$, $F_{\text{ДК}}$ та $Y_{\text{ДК}}$ — це мантий відповідних чисел у відповідних формах):

$R_{\text{ДК}} = F_{\text{ДК}} + Y_{\text{ДК}}$. Визначемо кожен доданок:

$F_{\text{ДК}}$:

$$\begin{array}{r} 1,010100 \\ + 0,000001 \\ \hline 1,010101 \end{array}$$

$$F_{\text{ДК}} = 1,010101$$

$$Y_{\text{ДК}} = Y_{\text{ПК}} = 0,000000$$

Тут варто зазначити щодо нормалізації. Мантия $F_{\text{ДК}}$ вважається нормалізованою через те, що у старшому розряді стоїть 0, за умови, що від'ємне число подане в ДК. Щодо мантий $Y_{\text{ДК}}$, то вона хоч і не є нормалізованою, але домогтись нормалізації неможливо, через те, що всі її розряди дорівнюють 0 і операцію зсуву можна виконувати нескінченно — через це вважатимемо її *машинним нулем*.

Тому дані числа у форматі плаваючої комі в ДК вважаються нормалізованими:

$$\begin{array}{l} F_{\text{ДК}} = \boxed{0} \quad \boxed{1010} \quad \boxed{1} \quad \boxed{010101} \\ Y_{\text{ДК}} = \boxed{0} \quad \boxed{1010} \quad \boxed{0} \quad \boxed{000000} \end{array}$$

Тепер можна виконувати додавання мантис. Використовуватимемо модифіковані коди, які забезпечуть збереження знаку:

$$F_{\text{мод,ДК}} = 11,010101, Y_{\text{мод,ДК}} = 00,000000$$

Тоді $R_{\text{мод,ДК}}$:

$$\begin{array}{r} 11,010101 \\ + 00,000000 \\ \hline 11,010101 \end{array}$$

Тобто $R_{\text{мод,ДК}} = 11,010101$.

Щоб перевести число ДК назад в ПК, всі розряди, окрім знакових, інвертуються та до результату інвертування додається одиниця, якщо знаковий біт дорівнює 1. Якщо знаковий біт = 0, то число в ДК співпадає з числом в ПК. Так як у нас знаковий біт = 1, то:

Тоді $R_{\text{мод,ПК}}$:

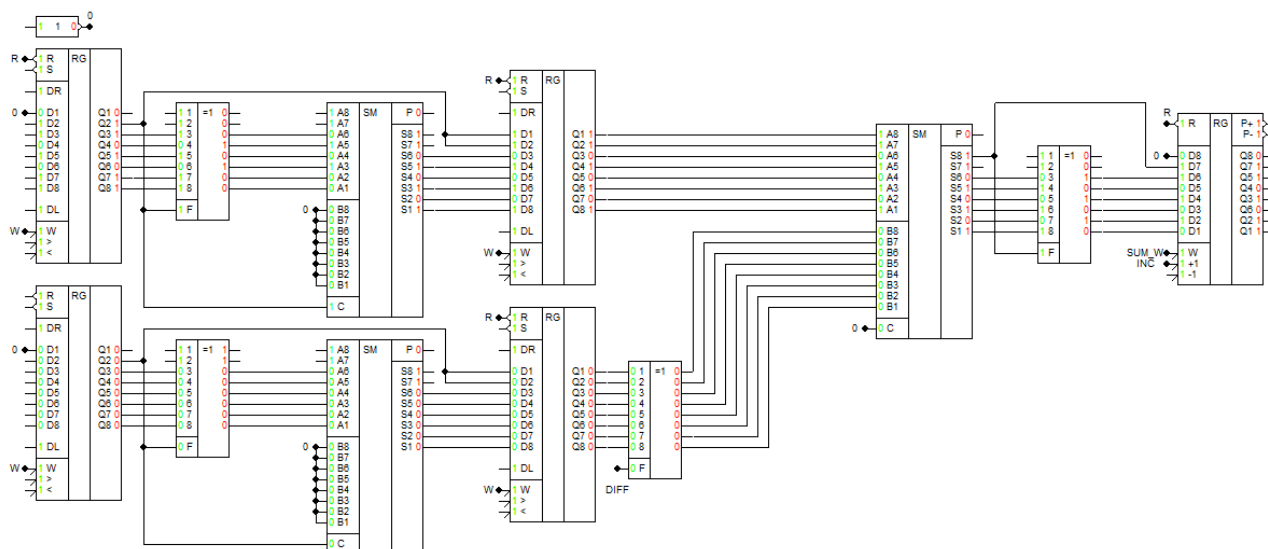
$$\begin{array}{r} 11,101010 \\ + 00,000001 \\ \hline 11,101011 \end{array}$$

Звідси $R_{\text{ПК}} = M_R = 1,101011$, а також $P_R = P_F = 1010$. Тоді число R у форматі з плаваючою комою:

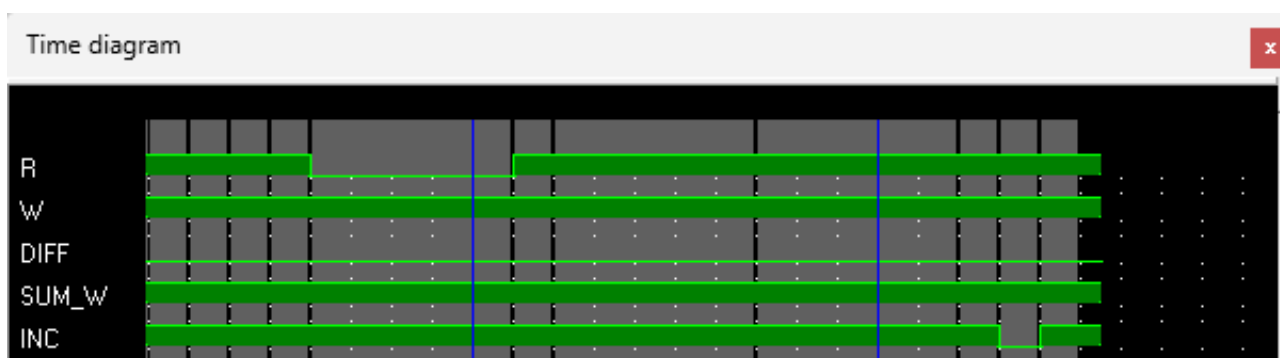
$$R = \boxed{0} \quad \boxed{1010} \quad \boxed{1} \quad \boxed{101011}$$

Що співпадає з числом F . Пояснення тому через те, що для представлення мантиси виділяється лише 7 бітів, один з яких є знаковим. Дана кількість бітів виявилась недостатньо точною для представлення суми двох чисел, порядки яких відрізняються на 7 бітів, що є дуже великим розривом. Тому під час сумування сформувалась природня похибка, тому радше буде записати не $=$, а $\approx$.

AFDK:



Часова діаграма:



Результат додавання збігається з попередньо обчисленим.

Висновок: Під час виконання цієї лабораторної роботи я поглибив та закріпив знання, отримані під час попередніх лабораторних занять. Зокрема, я повторив порядок виконання основних математичних операцій, а також набув практичних навичок роботи з числами з плаваючою комою. Окрім цього, я краще зрозумів особливості обробки таких чисел у контексті обчислень та можливі труднощі, що виникають під час їх використання.

Контрольні питання

1. *Як перейти від функціонального (змістовного) мікроалгоритму до структурного мікроалгоритму?*

Треба в блоках алгоритму замінити функції на оператори(сигнали), які призводять до їх виконання.

2. *Поясніть правила додавання та віднімання чисел в ОК і ДК.*

Числа в ОК додаються як звичайні числа у двійковій системі, правильний знак суми формується автоматично з урахуванням перенесення. Також, якщо присутнє переповнення, то воно циклічно переноситься в наймолодший розряд. У ДК все так само, тільки при переповненні відсутній циклічний переніс. Віднімання просто замінюється заміною числа на протилежне (інвертування всіх розрядів окрім знакових)

3. *Поясніть правила зсуву кодів в ОК і ДК.*

Якщо зсув логічний, то він для всіх кодів однаковий і відбувається зі зсувом розрядів та втратою розрядів, що вийшли за межі сітки. Пусті місця заповнюються нулями.

Якщо зсув арифметичний, то для кожного коду свої правила.

Для ПК: схожий на логічний зсув, окрім того, що знакові розряди не зсуюються.

Для ОК: зсув праворуч — молодший біт втрачається, старший розряд стає таким самим, як і знаковий, зсув ліворуч — всі розряди, включно із знаковими зсуюються: найстарший знаковий розряд поміщається у кінець (наймолодший) розряд.

Для ДК: зсув праворуч — такий самий, як і для ОК, зсув ліворуч — такий самий як для ОК, окрім того, що відсутній циклічний переніс старшого знакового розряду в молодший. Молодший натомість заповнюється нулем.

4. *Як можна виявити переповнення розрядної сітки при виконанні операцій з машинними кодами?*

Для виявлення переповнення в комп'ютерних системах часто використовують модифіковані машинні коди, в яких додатковий знаковий розряд ЗР2 знаходиться ліворуч від основного розряду ЗР1. Якщо відбувається переповнення розрядної сітки, то старший знаковий розряд завжди зберігає знак результату, а сам факт переповнення фіксується відмінністю знакових розрядів ЗР1 та ЗР2.

5. *Яким чином можна подати мікрооперації і мікроалгоритми?*

Мікрооперації — це найменші операції, які виконує процесор комп'ютера, щоб обробити інформацію. Мікрооперації можуть бути записані в таблицю і містити ім'я операції, адресний регістр, регістр даних, зсув та коментар. Мікроалгоритми — це послідовності мікрооперацій, які виконуються в певному порядку, при певних умовах процесором для виконання команд або функцій. Мікроалгоритми можуть бути представлені у вигляді блок-схем або тексту, а також можуть бути записані у вигляді таблиці. У таблицях використовують позначення станів, щоб вказати, які мікрооперації будуть виконані залежно від стану процесора та операндів.

6. *Форми подання чисел в ЕОМ.*

З фіксованою або плаваючою комою.

7. *Виконання операцій з числами в форматі з фіксованою комою.*

Числа із фіксованою комою подаються лише мантисою, без порядку. Це означає, що операції над числами в форматі з фіксованою комою виконуються так само, як і над їх мантисами.

8. *Етапи додавання чисел з плаваючою комою.*

Зрівняння порядків, додавання мантис, нормалізація результату, округлення (за потреби), формування результату.

9. *Дайте означення мікрооперації та мікроалгоритму?*

Мікрооперація — програма на спеціалізованій, апаратно-залежній мові програмування, що реалізує керування процесором в системах з мікропрограмним керуванням.

Мікроалгоритм — послідовність мікрооперацій, що приводить до виконання операцій в залежності від зовнішніх аргументів та внутрішніх станів, називають мікроалгоритмом.

10. *Схарактеризувати основні етапи проектування пристрою для виконання операції.*

- (а) Вивчити математичну основу та доведення певної операції над числом.
- (б) Побудувати операційну схему для певної операції.
- (в) Розробити змістовний мікроалгоритм.
- (г) Розробити функціонально-структурний мікроалгоритм, замінюючи мікрооперації сигналами.
- (д) Побудувати функціональну схему.
- (е) Побудувати логічну схему в засобах побудови логічних пристроїв.

11. *Як визначити необхідну тривалість керуючих сигналів?*

Керуючий сигнал повинен бути достатньо довгим для того, щоб встиг прийти сигнал зворотного зв'язку.

12. *Як перейти від операційної схеми до функціональної?*

На основі операційної схеми та ФС мікроалгоритму та на основі базису доступних операційних пристроїв (регістр, суматор і т.д.) побудувати схему, яка детально відображає особливості та структуру логічної, реальної схеми.

13. *Як розрахувати розрядність вузлів операційного пристрою?*

Все залежить від різних способів виконання операцій — це зумовлюється різними чинниками, такі як особливість методу, збільшення точності, запобіганню переповнення, тощо.